

Chapter 68: NgRx

Scenario 1: Do we even need NgRx for this feature?

Situation: A team is building a "notification bell" feature — a dropdown showing the last 20 notifications, a read/unread count, and a mark-as-read action. One engineer immediately proposes a full NgRx feature slice with actions, reducer, effects, and selectors. Another argues a simple signal-based service would do the job in a fraction of the code.

Question: How do you decide whether a feature genuinely needs NgRx (Store) versus a local signal-based service? What criteria would you apply to this notification bell specifically?

Answer: NgRx earns its complexity budget when you have several of these traits together: state shared across many unrelated parts of the tree, state that must survive navigation/route destruction, complex derived state that benefits from memoized selectors, non-trivial async orchestration (race conditions, retries, debouncing, cancellation), a need for time-travel debugging/auditing, or strict testability requirements around business rules that multiple teams touch.

For the notification bell: it's a single, self-contained widget. State (list, unread count) is only consumed by that component and its dropdown. There's no complex cross-cutting derivation, no multi-step async saga — just "fetch, mark read". A signal-based service is far cheaper to write, test, and reason about, and it scales down without ceremony.

```
// signal-based service: right-sized for this feature
@Injectable({ providedIn: 'root' })
export class NotificationStore {
  private http = inject(HttpClient);

  private readonly _notifications = signal<Notification[]>([]);
  private readonly _loading = signal(false);

  readonly notifications = this._notifications.asReadonly();
  readonly unreadCount = computed(
    () => this._notifications().filter(n => !n.read).length
  );
  readonly loading = this._loading.asReadonly();

  load() {
    this._loading.set(true);
    this.http.get<Notification[]>('/api/notifications').subscribe({
      next: list => this._notifications.set(list),
      complete: () => this._loading.set(false),
    });
  }

  markRead(id: string) {
    this._notifications.update(list =>
      list.map(n => (n.id === id ? { ...n, read: true } : n))
    );
    this.http.patch(`/api/notifications/${id}`, { read: true }).subscribe();
  }
}
```

```
}
```

I'd only escalate to NgRx if the notification count needs to be reactively consumed by five unrelated modules (nav bar, dashboard widget, settings page audit trail), or if we need to replay/undo notification state for support debugging. The rule of thumb I use: "NgRx should be the answer to a state-sharing/orchestration problem you've already confirmed exists, not a default architecture."

Interviewer intent: Tests whether the candidate reaches for heavy tooling reflexively or actually evaluates cost/benefit — a strong signal of architectural maturity.

Scenario 2: Selector recomputing on every dispatch despite memoization

Situation: A `cartTotal` selector is defined with `createSelector`, and the team notices — via Redux DevTools performance monitor — that it recomputes on *every* action dispatched anywhere in the app, including unrelated ones like `[Router] Navigation`. The projector function is expensive (iterates hundreds of line items), so this is causing visible jank.

Question: Why would a memoized selector recompute on unrelated actions, and how do you fix it?

Answer: `createSelector`'s memoization is based on reference equality of its *input selectors' results*, not on which action was dispatched. If any of the input selectors return a **new reference** each time state changes — even for unrelated slices — the memoized projector reruns. The classic culprit is an input selector that does inline derivation (e.g., `state => state.cart.items.filter(...)` or `.map(...)`) instead of a plain reference read. Every action that touches the store causes the root reducer to run; if any parent reducer returns a new object reference for a slice touched by an inline-deriving input selector, that "new reference" propagates even though the underlying data is unchanged.

Another common cause: the selector is created **inside a component constructor or a factory function called per-instance**, so each instance has its own memoization cache that gets invalidated by props/inputs changing, defeating the purpose entirely.

Fix: keep input selectors as pure "slice getters" (no computation), and push all computation into the outer projector, which is memoized as a whole.

```
// BAD: input selector does work, returns new array reference each state change
const selectCartTotal = createSelector(
  (state: AppState) => state.cart.items.filter(i => i.active), // new ref every time!
  (activeItems) => activeItems.reduce((sum, i) => sum + i.price * i.qty, 0)
);

// GOOD: input selectors are plain reference reads; work happens once, in the projector
const selectCartItems = createSelector(
  (state: AppState) => state.cart,
  (cart) => cart.items
);

const selectCartTotal = createSelector(
  selectCartItems,
  (items) => items
    .filter(i => i.active)
```

```
.reduce((sum, i) => sum + i.price * i.qty, 0)
);
```

I'd also verify the selector is declared at module scope (a singleton), not re-instantiated per component, and double check the reducer isn't spreading unrelated slices unnecessarily (e.g., `return { ...state, cart: { ...state.cart } }` on every action, even actions cart doesn't care about — that `{...state.cart}` line alone breaks memoization). Finally, I'd verify with `createSelector`'s underlying `defaultMemoize` — each selector keeps only a cache of size 1, so if the same selector is called with alternating different state slices (e.g., used against two different feature instances), you get cache thrashing; in that case `createSelectorFactory` with a larger cache, or per-instance parameterized selectors via a selector-factory function, is the fix.

Interviewer intent: Confirms deep, correct understanding of how NgRx memoization actually works (reference equality of inputs), not just "selectors are memoized" surface knowledge.

Scenario 3: Effect causing an infinite dispatch loop

Situation: After introducing a `syncSettings$` effect that listens for `settingsUpdated` and calls an API, then dispatches `settingsUpdated` again on success to "confirm" the update, the app freezes and DevTools shows thousands of `settingsUpdated` actions per second.

Question: What's wrong with this effect, and how do you redesign it to avoid a feedback loop while still confirming success?

Answer: The effect listens for the exact same action it dispatches on success, so every successful API call re-triggers itself — a self-feeding loop. This is one of the most common effect bugs: an effect's output action type overlaps with (or is broader than) its input `ofType` filter.

```
// BAD: dispatches the same action type it listens for
export const syncSettings$ = createEffect(
  (actions$ = inject(Actions), api = inject(SettingsApi)) =>
    actions$.pipe(
      ofType(SettingsActions.settingsUpdated),
      switchMap(({ settings }) =>
        api.save(settings).pipe(
          map(() => SettingsActions.settingsUpdated({ settings }))) // loops forever!
        )
      )
    )
);
```

The fix is to separate the "intent" action from the "result" action(s) — a strict command/event distinction. Use `createActionGroup` to model `update` (command from the UI), `updateSuccess`, and `updateFailure` (results from the effect), so the effect's input and output types never overlap.

```
export const SettingsActions = createActionGroup({
  source: 'Settings',
  events: {
    update: props<{ settings: Settings }>(),
    'update success': props<{ settings: Settings }>(),
    'update failure': props<{ error: string }>(),
  }
});
```

```

    },
  });

export const syncSettings$ = createEffect(
  (actions$ = inject(Actions), api = inject(SettingsApi)) =>
    actions$.pipe(
      ofType(SettingsActions.update),
      switchMap(({ settings }) =>
        api.save(settings).pipe(
          map((saved) => SettingsActions.updateSuccess({ settings: saved })),
          catchError((error) => of(SettingsActions.updateFailure({ error: error.message })))
        )
      )
    )
);

```

Beyond the type-overlap fix, I'd add two defensive layers for production code: (1) an ESLint rule or code-review checklist item that flags any effect whose `ofType` filter set intersects with an action type it also dispatches, and (2) in genuinely recursive-by-design flows (e.g., polling), always gate with an explicit termination condition (`takeUntil`, a max-retry count, or a debounce) rather than relying on "it shouldn't loop in practice."

Interviewer intent: Tests whether the candidate has actually debugged a production infinite-loop effect and understands the command/event separation discipline that prevents it structurally.

Scenario 4: Normalizing nested/relational state (orders → line items → products)

Situation: The app stores `Order[]`, where each order has an embedded array of `LineItem[]`, and each line item embeds a full `Product` object. Updating a product's price (e.g., after a price-list refresh) requires walking every order and every line item to patch the denormalized copies, and this is a recurring source of stale-data bugs.

Question: How would you redesign this state shape to avoid the update-fan-out problem, and what does the reducer/selector code look like afterward?

Answer: Denormalized, deeply nested state means a single entity update requires a deep, repetitive rewrite everywhere that entity is embedded — expensive and bug-prone (miss one nesting path and you have inconsistent state). The fix is normalization: store each entity type in its own flat dictionary keyed by ID, and have orders/line items reference other entities by ID only, similar to a relational database's foreign-key model. NgRx's `@ngrx/entity` (`createEntityAdapter`) is the standard tool for each flat table, and selectors reconstitute the "joined" view on demand.

```

// normalized shape
interface ProductsState extends EntityState<Product> {}
interface LineItemsState extends EntityState<LineItem> {} // LineItem: { id, orderId,
productId, qty }
interface OrdersState extends EntityState<Order> {}      // Order: { id, lineItemIds:
string[] }

const productsAdapter = createEntityAdapter<Product>();

```

```

const lineItemsAdapter = createEntityAdapter<LineItem>();
const ordersAdapter = createEntityAdapter<Order>();

// updating a product price touches ONE table, not every order
export const productsReducer = createReducer(
  productsAdapter.getInitialState(),
  on(ProductActions.priceUpdated, (state, { productId, price }) =>
    productsAdapter.updateOne({ id: productId, changes: { price } }, state)
  )
);

// selectors "join" normalized tables back into a view model
export const selectOrderWithDetails = (orderId: string) => createSelector(
  selectOrderEntities,
  selectLineItemEntities,
  selectProductEntities,
  (orders, lineItems, products) => {
    const order = orders[orderId];
    if (!order) return undefined;
    return {
      ...order,
      lineItems: order.lineItemIds
        .map(id => lineItems[id])
        .filter(Boolean)
        .map(li => ({ ...li, product: products[li.productId] })),
    };
  }
);

```

Tradeoffs: normalization adds a "join" cost at read time (mitigated by memoized selectors — the join only reruns when one of the three tables actually changes) and requires more careful ID-management on the write side (e.g., an order create action must also create/link line items). But it eliminates fan-out updates entirely, makes each entity's mutations trivially testable in isolation, and plays nicely with entity adapter's built-in `updateOne` / `updateMany` / `upsertMany` for bulk syncs (e.g., a price-list refresh becomes one `upsertMany` call).

Interviewer intent: Tests whether the candidate can apply relational-database normalization principles to client-side state — a common signal of scaling experience beyond toy apps.

Scenario 5: Optimistic update with rollback

Situation: A Kanban board lets users drag a card between columns. The team wants the UI to update instantly (optimistic), but if the backend rejects the move (e.g., permission changed mid-drag), the card must snap back to its original column with a toast error.

Question: Design the actions, reducer, and effect for an optimistic move-with-rollback flow in NgRx.

Answer: The key design principle: the reducer applies the optimistic change **synchronously on the command action** (not waiting for the effect), and the effect performs the API call; on failure, it dispatches a rollback action carrying enough information (the *previous* state, or an inverse operation) to precisely undo the optimistic change — never "refetch everything," which is slow and can race with other in-flight edits.

```

export const CardActions = createActionGroup({
  source: 'Kanban Card',
  events: {
    'Move Card': props<{ cardId: string; fromColumnId: string; toColumnId: string }>(),
    'Move Card Success': props<{ cardId: string }>(),
    'Move Card Failure': props<{ cardId: string; fromColumnId: string; toColumnId: string;
error: string }>(),
  },
});

export const cardsReducer = createReducer(
  initialState,
  // optimistic apply
  on(CardActions.moveCard, (state, { cardId, toColumnId }) =>
    cardsAdapter.updateOne({ id: cardId, changes: { columnId: toColumnId } }, state)
  ),
  // success: no-op, state is already correct
  on(CardActions.moveCardSuccess, (state) => state),
  // rollback: revert to the original column
  on(CardActions.moveCardFailure, (state, { cardId, fromColumnId }) =>
    cardsAdapter.updateOne({ id: cardId, changes: { columnId: fromColumnId } }, state)
  )
);

export const moveCard$ = createEffect(
  (actions$ = inject(Actions), api = inject(BoardApi), toast = inject(ToastService)) =>
    actions$.pipe(
      ofType(CardActions.moveCard),
      concatMap(({ cardId, fromColumnId, toColumnId }) =>
        api.moveCard(cardId, toColumnId).pipe(
          map(() => CardActions.moveCardSuccess({ cardId })),
          catchError((error) => {
            toast.error('Could not move card - reverting.');
```

Design notes: I use `concatMap` (not `switchMap` / `mergeMap`) so moves for the *same board* are serialized in the order the user made them — important because optimistic UI + out-of-order network responses can otherwise leave the board in a state the user never actually produced. If moves for *different* cards should be independent and concurrent, key the concatenation per-card with `groupBy(cardId)` + `mergeMap(group => group.pipe(concatMap(...)))`. I also always carry the "undo data" (`fromColumnId`) in the failure action itself rather than looking it up from current state in the reducer at rollback time — current state may have changed again by the time the failure arrives, so the rollback must be self-contained.

Interviewer intent: Tests whether the candidate can design robust optimistic-UI flows that survive out-of-order responses and don't silently corrupt state — a very common real-world NgRx interview topic.

Scenario 6: Testing a reducer in isolation

Situation: A code reviewer rejects a PR because the only test for a new reducer case is an integration test that renders a component and asserts on DOM text. They ask for a proper unit test of the reducer function itself.

Question: Write a focused unit test for the `moveCardFailure` reducer case above, and explain why reducer tests should avoid the TestBed entirely.

Answer: Reducers are pure functions — `(state, action) => newState` — with no dependencies on Angular's DI, HTTP, or the DOM. Testing them via TestBed/component rendering adds enormous overhead and indirection to verify what is fundamentally simple, deterministic logic. The correct approach is to call the reducer function directly with a hand-built input state and assert on the exact output state, without any Angular test harness.

```
import { cardsReducer } from './cards.reducer';
import { CardActions } from './card.actions';
import { cardsAdapter } from './cards.adapter';

describe('cardsReducer', () => {
  it('rolls back the column on moveCardFailure', () => {
    const initialState = cardsAdapter.setAll(
      [{ id: 'card-1', columnId: 'in-progress', title: 'Fix bug' }],
      cardsAdapter.getInitialState()
    );

    // simulate the optimistic move having already happened
    const optimisticState = cardsAdapter.updateOne(
      { id: 'card-1', changes: { columnId: 'done' } },
      initialState
    );

    const action = CardActions.moveCardFailure({
      cardId: 'card-1',
      fromColumnId: 'in-progress',
      toColumnId: 'done',
      error: 'Forbidden',
    });

    const result = cardsReducer(optimisticState, action);

    expect(result.entities['card-1']?.columnId).toBe('in-progress');
  });

  it('returns the same reference for unrelated actions (memoization-friendly)', () => {
    const state = cardsAdapter.getInitialState();
    const result = cardsReducer(state, { type: 'Unrelated Action' } as any);
    expect(result).toBe(state);
  });
});
```

I'd emphasize two things in review: first, no `TestBed.configureTestingModule` is needed — the reducer is imported and called like any plain function, which makes the test run in milliseconds and fail with a precise diff. Second, I explicitly test the "unrelated action returns the same reference" case, because that's exactly the property that keeps downstream selectors memoized (tying back to Scenario 2) — a regression there is easy to introduce accidentally (e.g., someone adds a default case that spreads state unconditionally) and a snapshot/reference-equality test catches it immediately.

Interviewer intent: Confirms the candidate understands reducers as pure functions deserving pure-function tests, and connects testing discipline to the memoization concerns raised elsewhere in the store.

Scenario 7: Testing an effect with marble testing

Situation: The `moveCard$` effect from Scenario 5 needs a unit test that verifies: on API success it emits `moveCardSuccess`, and on API failure it emits `moveCardFailure` — without hitting a real network.

Question: Write a test for this effect. Would you use marble testing or a simpler Promise/Subject-based approach, and why?

Answer: Both are valid; marble testing (via `provideMockActions` + RxJS `TestScheduler`) is the traditional NgRx approach and is excellent for effects with complex timing (debounce, race, retry-with-backoff), where asserting on *frame-by-frame* emission timing matters. For effects whose logic is really "map success/error to an action" without complex timing, a simpler `provideMockActions` with a plain `Actions` stream and `firstValueFrom / toArray` is more readable and just as rigorous — I'd reserve marbles for when timing itself is the thing under test.

```
// Simple approach – no marbles needed, timing isn't the point here
describe('moveCard$ effect', () => {
  let actions$: Observable<Action>;
  let api: jasmine.SpyObj<BoardApi>;

  function setup(source$: Observable<Action>) {
    api = jasmine.createSpyObj('BoardApi', ['moveCard']);
    TestBed.configureTestingModule({
      providers: [
        provideMockActions(() => source$),
        { provide: BoardApi, useValue: api },
        { provide: ToastService, useValue: { error: jasmine.createSpy() } },
      ],
    });
  }

  it('dispatches moveCardSuccess on API success', (done) => {
    const action = CardActions.moveCard({ cardId: 'c1', fromColumnId: 'todo', toColumnId: 'done' });
    setup(of(action));
    api.moveCard.and.returnValue(of(void 0));

    TestBed.runInInjectionContext(() => {
      moveCard$(TestBed.inject(Actions), api,
        TestBed.inject(ToastService)).subscribe((result) => {
        expect(result).toEqual(CardActions.moveCardSuccess({ cardId: 'c1' }));
      });
    });
  });
});
```



```

        done();
    });
});
});

it('dispatches moveCardFailure on API error', (done) => {
    const action = CardActions.moveCard({ cardId: 'c1', fromColumnId: 'todo', toColumnId: 'done' });
    setup(of(action));
    api.moveCard.and.returnValue(throwError(() => new Error('Forbidden')));

    TestBed.runInInjectionContext(() => {
        moveCard$(TestBed.inject(Actions), api,
        TestBed.inject(ToastService)).subscribe((result) => {
            expect(result.type).toBe(CardActions.moveCardFailure.type);
            done();
        });
    });
});
});
});
});

```

If instead I were testing something like a search-typeahead effect using `debounceTime(300)` + `switchMap`, I'd reach for `TestScheduler.run(({ hot, cold, expectObservable }) => ...)` marble syntax specifically because I need to assert that keystrokes within the debounce window are collapsed and that an in-flight request is cancelled by `switchMap` when a newer keystroke arrives — properties that are about *time*, which marbles express far more precisely than sequential `done()` callbacks.

Interviewer intent: Tests whether the candidate over-uses marble testing reflexively or picks the right tool for the effect's actual complexity — and whether they understand what marbles are actually good for (timing assertions).

Scenario 8: Testing a memoized selector

Situation: QA found that `selectCartTotal` (Scenario 2, fixed version) returned an incorrect total when items had a `qty` of 0. The fix needs a unit test, but the reviewer also wants a test proving the selector doesn't recompute unnecessarily.

Question: Write tests for both the selector's correctness and its memoization behavior.

Answer: Selector unit tests should call `.projector` directly for pure logic verification (fast, no store needed), and separately assert on memoization using the selector's `release` /call-count behavior or by spying on the projector.

```

describe('selectCartTotal', () => {
    it('computes the correct total, ignoring inactive items', () => {
        const items = [
            { id: '1', active: true, price: 10, qty: 2 },
            { id: '2', active: true, price: 5, qty: 0 },
            { id: '3', active: false, price: 100, qty: 5 },
        ];
        // .projector runs the pure computation without needing full store state
    });
});

```

```

    expect(selectCartTotal.projector(items)).toBe(20);
  });

  it('memoizes and does not recompute for an unrelated state slice change', () => {
    const state1 = { cart: { items: [{ id: '1', active: true, price: 10, qty: 1 }] }, user:
    { name: 'A' } } as any;
    const state2 = { ...state1, user: { name: 'B' } }; // unrelated slice changed

    const spy = spyOn(selectCartTotal, 'release').and.callThrough();

    const result1 = selectCartTotal(state1);
    const result2 = selectCartTotal(state2);

    expect(result1).toBe(result2); // same computed value, and ideally same reference for
    objects
    expect(selectCartItems(state1)).toBe(selectCartItems(state2)); // cart slice reference
    unchanged
  });
});

```

The correctness test targets `.projector` directly — this is the honest unit boundary for "given these inputs, what's the output," independent of memoization machinery. The memoization test asserts on the *input* selector's reference stability across two state objects that differ only in an unrelated slice, which is really what "the projector didn't rerun" implies (NgRx doesn't expose a public recompute counter, so asserting input-reference stability, or spying on the projector function itself before wrapping in `createSelector`, are the two practical techniques). I'd call out in review that testing selectors this way catches regressions like the Scenario 2 bug (an input selector doing `.filter()` inline) before it ever reaches DevTools performance profiling.

Interviewer intent: Tests whether the candidate can write tests that actually validate memoization semantics rather than just the output value — a subtlety many engineers miss.

Scenario 9: Migrating a feature from classic NgRx Store to NgRx Signal Store

Situation: The "user profile" feature currently uses classic `@ngrx/store` with actions, a reducer, effects, and selectors — about 150 lines for what is fundamentally CRUD on a single entity plus one async load. Leadership wants to modernize incrementally to `@ngrx/signals` (Signal Store) without breaking the rest of the app that still uses classic Store for other features.

Question: Walk through the migration: what does the Signal Store version look like, and how do you keep both coexisting during the transition?

Answer: `@ngrx/signals`' `signalStore` is designed to coexist with classic `@ngrx/store` in the same app — they're independent packages and can be adopted feature-by-feature. The migration pattern: replace actions/reducer/selectors/effects for that one feature with `withState`, `withComputed`, and `withMethods` (using `rxMethod` for async work), while leaving the global `StoreModule / provideStore` and other feature slices untouched.

```

// BEFORE (classic): actions.ts, reducer.ts, effects.ts, selectors.ts – ~150 lines total
// AFTER: one file, signal store

```



```

})
export class ProfileComponent {
  protected store = inject(ProfileStore);
  constructor() {
    this.store.load(inject(AuthService).currentUserId());
  }
}

```

Coexistence strategy: I keep `provideStore()` and `provideEffects()` at the root exactly as they were for every feature not yet migrated; the new `ProfileStore` is just another injectable and doesn't need to be registered in the root `Store`. There's no requirement to migrate everything atomically. What I do watch for: (1) don't let the two paradigms leak into each other — a classic effect shouldn't directly `patchState` a signal store, keep a clean boundary, communicate via well-defined APIs if they must interact; (2) Redux DevTools integration for Signal Store requires explicit `withDevtools()` (from `@ngrx/signals/devtools` in recent versions) if you still want time-travel visibility during the transition; (3) re-verify testability — Signal Store methods are tested by instantiating the store via `TestBed` and calling methods directly, asserting on the exposed signals, which is arguably simpler than the four-file classic test suite it replaced.

Interviewer intent: Tests knowledge of the newest NgRx Signal Store API surface and whether the candidate can plan a realistic, incremental (not big-bang) migration in a real codebase.

Scenario 10: Debugging with Redux DevTools time-travel

Situation: A user reports that after applying three filters on a data table (category, price range, then sort), the table shows results consistent with only the *first* filter, even though the UI chips show all three as active. The bug is intermittent and hard to reproduce locally.

Question: Walk through how you'd use Redux DevTools' time-travel and action log to root-cause this, and what in the codebase you'd suspect first.

Answer: I'd start by reproducing the bug once, then open Redux DevTools and inspect the action log in order: `filterByCategory`, `filterByPriceRange`, `sortResults`. Time-travel lets me click each action in sequence and inspect the *state diff* pane after each — this immediately tells me whether the state itself became wrong at some step, or whether the state is correct but a selector/component is reading a stale slice.

Two concrete checks I'd do in DevTools:

1. **Jump to the state right after `filterByPriceRange`.** If `state.table.category` and `state.table.priceRange` are both correctly populated there, the reducer is fine — the bug is downstream (selector or component).
2. **Jump to the state right after `sortResults`** and diff it against the previous state. If the diff shows category/priceRange fields reverting to defaults, the `sortResults` reducer case is spreading from the *wrong* base object — e.g., closing over stale `initialState` instead of the current `state` parameter, or a case using `{ ...initialState, sort }` instead of `{ ...state, sort }` (a very common copy-paste bug when reducer cases are written by cloning an earlier case).

```
// the bug, once found – classic copy/paste error
on(TableActions.sortResults, (state, { sort }) => ({
  ...initialState, // BUG: should be ...state – wipes category & priceRange
  sort,
})),
```

Time-travel is especially valuable here because it turns an intermittent, hard-to-reproduce bug into a static, inspectable artifact — once the action log is captured, I can step through deterministically without needing to reproduce the race condition again. I'd also enable the "action sanitizer"/"state sanitizer" DevTools options to redact large payloads if the table data is large, so the diff view stays readable, and use the DevTools "Jump" (not just "Skip") feature to actually re-derive the live app state at that point, so the UI reflects exactly what the reducer produced — confirming whether the *component* also has a bug (e.g., a stale selector subscription) independent of the store.

Interviewer intent: Tests practical, hands-on DevTools debugging skill and whether the candidate can localize a bug to reducer vs. selector vs. component using time-travel rather than guesswork.

Scenario 11: Entity adapter for CRUD-heavy state (admin table)

Situation: An admin screen manages a list of ~2,000 "API keys" with inline create, rename, revoke (soft-delete), and bulk-select-and-revoke actions. The original implementation stored `apiKeys: ApiKey[]` in an array and did `state.apiKeys.findIndex(...)` + splice-style updates in every reducer case, which the team suspects is both slow and bug-prone for bulk operations.

Question: Refactor this to use `@ngrx/entity`'s `createEntityAdapter`, and show how bulk revoke is handled cleanly.

Answer: `createEntityAdapter` stores entities as `{ ids: string[], entities: { [id]: T } }`, turning lookups/updates from $O(n)$ array scans into $O(1)$ dictionary access, and provides battle-tested CRUD helper functions (`addOne`, `updateMany`, `removeMany`, `upsertMany`, etc.) that avoid the manual splice/findIndex bugs (off-by-one errors, forgetting immutability, mutating in place).

```
export interface Apikey {
  id: string;
  name: string;
  revoked: boolean;
  createdAt: string;
}

export const apikeysAdapter = createEntityAdapter<Apikey>({
  sortComparer: (a, b) => b.createdAt.localeCompare(a.createdAt),
});

export interface ApikeysState extends EntityState<Apikey> {
  selectedIds: string[];
  loading: boolean;
}

const initialState: ApikeysState = apikeysAdapter.getInitialState({
```

```

    selectedIds: [],
    loading: false,
  });

export const ApiKeyActions = createActionGroup({
  source: 'Api Keys',
  events: {
    'Load Success': props<{ keys: ApiKey[] }>(),
    Create: props<{ name: string }>(),
    'Create Success': props<{ key: ApiKey }>(),
    'Toggle Selected': props<{ id: string }>(),
    'Bulk Revoke': props<{ ids: string[] }>(),
    'Bulk Revoke Success': props<{ ids: string[] }>(),
  },
});

export const apiKeysReducer = createReducer(
  initialState,
  on(ApiKeyActions.loadSuccess, (state, { keys }) =>
    apiKeysAdapter.setAll(keys, state)
  ),
  on(ApiKeyActions.createSuccess, (state, { key }) =>
    apiKeysAdapter.addOne(key, state)
  ),
  on(ApiKeyActions.toggleSelected, (state, { id }) => ({
    ...state,
    selectedIds: state.selectedIds.includes(id)
      ? state.selectedIds.filter(x => x !== id)
      : [...state.selectedIds, id],
  })),
  on(ApiKeyActions.bulkRevokesSuccess, (state, { ids }) =>
    apiKeysAdapter.updateMany(
      ids.map(id => ({ id, changes: { revoked: true } })),
      { ...state, selectedIds: [] }
    )
  )
);

// generated selectors – no manual boilerplate
export const { selectAll, selectEntities, selectIds } = apiKeysAdapter.getSelectors();
export const selectApiKeysState = createFeatureSelector<ApiKeysState>('apiKeys');
export const selectAllApiKeys = createSelector(selectApiKeysState, selectAll);
export const selectSelectedKeys = createSelector(
  selectApiKeysState,
  selectEntities,
  (state, entities) => state.selectedIds.map(id => entities[id!]).filter(Boolean)
);

```

Bulk revoke's effect fires one batched API call and one `updateMany` — never a loop of individual `updateOne` dispatches, which would both thrash change detection and be harder to roll back atomically on partial failure.

```
export const bulkRevoke$ = createEffect(
  (actions$ = inject(Actions), api = inject(ApiKeysApi)) =>
    actions$.pipe(
      ofType(ApiKeyActions.bulkRevoke),
      exhaustMap(({ ids }) =>
        api.revokeMany(ids).pipe(
          map(() => ApiKeyActions.bulkRevokeSuccess({ ids }))
        )
      )
    )
);
```

I'd use `exhaustMap` here specifically because a bulk revoke shouldn't be re-triggered while one is already in flight (prevents double-submission from a fast double-click), unlike the Kanban move in Scenario 5 where `concatMap` was right for queuing distinct user-initiated moves.

Interviewer intent: Tests familiarity with `@ngrx/entity`'s API surface and whether the candidate picks the correct RxJS flattening operator per use case rather than defaulting to `switchMap` everywhere.

Scenario 12: Selector recomputation caused by an object literal in the component template

Situation: A performance audit shows a `selectFilteredProducts` selector recomputing on every change-detection cycle — not just on relevant dispatches — even though its reducer slice barely changes. The component uses `store.select(selectFilteredProducts({ category: this.category, maxPrice: this.maxPrice })))` inside the template via the async pipe, called as a parameterized selector factory.

Question: What's causing recomputation on every CD cycle here, and how do you fix a parameterized selector so it's actually memoized per parameter set?

Answer: Two things compound here. First, `selectFilteredProducts({ category, maxPrice })` — a selector *factory* called directly in the template — creates a brand-new selector instance (and therefore a brand-new memoization cache) on every template re-evaluation, since `{ category, maxPrice }` is a new object literal each time and the factory function itself runs fresh each call. The fix is to create the parameterized selector **once** (e.g., in the component class, recomputed only when the actual parameter values change) rather than inline in the template.

```
// BAD: factory called fresh in the template -> new selector, no memoization ever kicks in
// template: {{ (store.select(selectFilteredProducts({ category, maxPrice }))) | async) }}

// GOOD: parameterized selector factory defined once, memoized cache reused
export const selectFilteredProducts = (params: { category: string; maxPrice: number }) =>
  createSelector(
    selectAllProducts,
    (products) => products.filter(
      p => p.category === params.category && p.price <= params.maxPrice
    )
  );
```

```

@Component({ /* ... */ })
export class ProductListComponent {
  private store = inject(Store);
  category = input.required<string>();
  maxPrice = input.required<number>();

  // recreated only when inputs actually change, via computed + toSignal-friendly pattern
  private params = computed(() => ({ category: this.category(), maxPrice: this.maxPrice()
}));

  products = toSignal(
    toObservable(this.params).pipe(
      distinctUntilChanged((a, b) => a.category === b.category && a.maxPrice ===
b.maxPrice),
      switchMap((params) => this.store.select(selectFilteredProducts(params)))
    ),
    { initialValue: [] }
  );
}

```

Even with this fix, a factory-created selector's memoization cache is still typically size-1 per instance (`defaultMemoize`), so if the same selector instance were reused across wildly different parameter sets it would still thrash — that's expected and fine here because each component instance owns one selector instance for its own stable params. If instead I needed a *shared, cross-component* cache keyed by parameters (e.g., many list widgets filtering the same category), I'd reach for `createSelectorFactory` with a custom memoization function (e.g., a small LRU keyed by a serialized params string) rather than the default single-slot memoize. The core lesson stands regardless: never call a selector *factory* directly inside a template or a `computed` /effect body without ensuring the factory itself is invoked only when parameters truly change — otherwise "memoization" is theater.

Interviewer intent: Tests deep understanding of parameterized selector factories, a nuance that trips up even experienced NgRx developers — memoization only works if the selector instance persists.

Scenario 13: Effect cancellation and race conditions on rapid user input

Situation: A live search box dispatches `searchProducts({ query })` on every keystroke (after a 200ms debounce in the component). QA reports that typing "lap", pausing, then quickly changing to "desk" occasionally shows stale "lap"-related results flashing before "desk" results appear.

Question: Diagnose the race condition and fix the effect.

Answer: The likely cause is the effect using `mergeMap` (or worse, no flattening discipline at all) instead of `switchMap`, so both the "lap" and "desk" HTTP requests are in flight concurrently, and whichever response arrives last wins — if "lap"'s response is slower for some backend reason (cache miss, larger result set) it can resolve *after* "desk"'s and briefly overwrite the correct state.

```

// BAD: mergeMap lets both requests race; last response wins, not last request
export const search$ = createEffect(
  (actions$ = inject(Actions), api = inject(SearchApi)) =>

```



```

actions$.pipe(
  ofType(SearchActions.searchProducts),
  mergeMap(({ query }) =>
    api.search(query).pipe(map(results => SearchActions.searchSuccess({ results })))
  )
);

// GOOD: switchMap cancels the previous in-flight request when a new query arrives
export const search$ = createEffect(
  (actions$ = inject(Actions), api = inject(SearchApi)) =>
    actions$.pipe(
      ofType(SearchActions.searchProducts),
      switchMap(({ query }) =>
        api.search(query).pipe(
          map(results => SearchActions.searchSuccess({ query, results })),
          catchError(() => of(SearchActions.searchFailure({ query })))
        )
      )
    )
);

```

`switchMap` unsubscribes from the previous inner observable (cancelling the HTTP request via Angular's `HttpClient` teardown) the instant a new `searchProducts` action arrives, so only the most recent query's response can ever reach the reducer. I'd also add a defensive guard in the reducer itself — stamping the query on both the action and the resulting state, and having the component(selector ignore a `searchSuccess` whose `query` doesn't match the current input value — as a second line of defense in case the debounce window and the effect's cancellation still leave a tiny window (e.g., a request that started right before `switchMap` cancellation logic engages due to microtask ordering). For "search-as-you-type," `switchMap` is essentially always correct because only the latest query's result matters, unlike Scenario 5's `concatMap` (order matters, no result should be discarded) or Scenario 11's `exhaustMap` (ignore new triggers while one is in flight).

Interviewer intent: Tests the candidate's mental model of `switchMap` vs `mergeMap` vs `concatMap` vs `exhaustMap` applied correctly to a concrete race-condition bug, not just definitions recited from memory.

Scenario 14: State shape design — avoiding derived state duplication

Situation: A junior engineer's PR stores both `products: Product[]` and `filteredProducts: Product[]` in the same reducer slice, updating `filteredProducts` manually inside every reducer case that touches `products` or the active filter.

Question: Why is storing `filteredProducts` in the reducer state an anti-pattern, and how should it be modeled instead?

Answer: `filteredProducts` is *derived* state — fully computable from `products` and the current filter criteria. Storing it as its own state field means every reducer case that changes either input must remember to also recompute and re-store the derived value, which is both redundant work and a guaranteed source of drift bugs (someone adds a new action that changes `products` but forgets to update `filteredProducts`, and now the two are inconsistent). The NgRx principle here mirrors general reactive-state-management best practice:

store the minimal source-of-truth state, derive everything else via selectors, which are automatically recomputed (and memoized) whenever their true inputs change.

```
// BAD
interface ProductsState {
  products: Product[];
  filter: { category: string };
  filteredProducts: Product[]; // derived, but stored & manually kept in sync
}

on(ProductActions.setFilter, (state, { filter }) => ({
  ...state,
  filter,
  filteredProducts: state.products.filter(p => p.category === filter.category), // easy to
  forget elsewhere
})))

// GOOD
interface ProductsState {
  products: Product[];
  filter: { category: string };
}

export const productsReducer = createReducer(
  initialState,
  on(ProductActions.setFilter, (state, { filter }) => ({ ...state, filter })),
  on(ProductActions.loadSuccess, (state, { products }) => ({ ...state, products })))
);

// derived via selector – always correct, automatically recomputed only when inputs change
export const selectFilteredProducts = createSelector(
  (s: AppState) => s.products.products,
  (s: AppState) => s.products.filter,
  (products, filter) => products.filter(p => p.category === filter.category)
);
```

The only time I'd consider persisting a derived value in state is when its computation is genuinely expensive (say, an $O(n^2)$ matching algorithm) *and* it needs to survive being recomputed on every selector call regardless of memoization overhead — even then, I'd model it as an explicit cache with its own invalidation action, not silently duplicate it alongside the source data as if it were independent truth. The litmus test I use in review: "if I deleted this field and recomputed it from the rest of state, would I get the same answer?" If yes, it doesn't belong in the reducer state.

Interviewer intent: Tests whether the candidate internalizes "derive, don't duplicate" as a state-shape principle, catching a very common junior mistake.

Scenario 15: createFeature for reducing selector/reducer boilerplate

Situation: A codebase still manually writes `createFeatureSelector`, then five individually-exported selectors, alongside the reducer, for every feature — about 30 lines of repetitive selector wiring per feature, multiplied across 12 features.

Question: How does `createFeature` reduce this boilerplate, and what does it look like applied to the `apiKeys` feature from Scenario 11?

Answer: `createFeature` bundles a feature's reducer with its `initialState`'s key name and auto-generates a full set of "select every top-level property" selectors, plus the feature selector itself, in one call — eliminating the need to hand-write `createFeatureSelector` and a selector per field.

```
export const apiKeysFeature = createFeature({
  name: 'apiKeys',
  reducer: apiKeysReducer,
  extraSelectors: ({ selectApiKeysState, selectSelectedIds }) => ({
    selectAllKeys: createSelector(selectApiKeysState,
      apiKeysAdapter.getSelectors().selectAll),
    selectSelectedKeys: createSelector(
      selectApiKeysState,
      selectSelectedIds,
      (state, ids) => ids.map(id => state.entities[id!]).filter(Boolean)
    ),
  }),
});

// auto-generated: selectApiKeysState, selectIds, selectEntities, selectSelectedIds,
// selectLoading, ...
export const {
  name,
  reducer,
  selectApiKeysState,
  selectSelectedIds,
  selectLoading,
  selectAllKeys,
  selectSelectedKeys,
} = apiKeysFeature;
```

```
// registration – no manual createFeatureSelector needed elsewhere in the app
export const provideApiKeysFeature = () => provideState(apiKeysFeature);
```

This collapses what used to be a `selectors.ts` file (`createFeatureSelector` + N selectors) into `extraSelectors` co-located with the reducer definition, which also improves discoverability — a new engineer opens one file and sees state shape, reducer logic, and every selector for the feature together, rather than hunting across three files. The tradeoff: `extraSelectors`' auto-inferred top-level selectors are only useful for direct property reads; anything requiring genuine cross-slice composition (joining with another feature) still needs a `createSelector` written the traditional way, typically in a separate file, since `createFeature` intentionally scopes itself to one feature's own state slice.

Interviewer intent: Tests whether the candidate is current on modern NgRx idioms (`createFeature`, `createActionGroup`) versus only knowing the older, more verbose API — a good "how current is your NgRx knowledge" probe.

Scenario 16: Root cause of "Cannot read properties of undefined" after lazy-loaded feature state isn't registered yet

Situation: A component injected in a lazy-loaded route immediately calls `store.select(selectOrdersState)` in its constructor and crashes with `Cannot read properties of undefined (reading 'orders')` intermittently — only on a hard refresh directly to that route, never on in-app navigation.

Question: What's the root cause, and how do you fix it robustly?

Answer: On a hard refresh directly into a lazy route, the feature's `provideState(ordersFeature)` (registered via the route's providers or a route-level `provideState` call) may not have completed registration *synchronously* before the component's constructor selector subscription first emits, depending on how/where the feature is provided (e.g., if it's provided asynchronously via a route resolver or a dynamically imported providers array that hasn't resolved yet) — or, more commonly, the selector is written against a feature-selector chain that assumes intermediate slices exist without guarding for the reducer's *own* initial state being incomplete relative to what the selector expects (e.g., an app-level global reducer runs before the feature reducer registers, and some other selector composes both).

The robust fixes, in order of preference: (1) always let `createFeature` / `createFeatureSelector` supply the initial state guarantee — NgRx returns `undefined` for a feature selector only if the feature truly isn't registered yet, so the real fix is ensuring `provideState` for the feature is part of the route's static `providers` array (resolved before component construction) rather than lazily provided inside an async factory; (2) defensively guard any composed/cross-feature selector with optional chaining and a sensible fallback; (3) prefer reading via the `Store`'s emitted `Observable` / signal in the template (`store.selectSignal(...)` combined with `@if`) rather than dereferencing store state synchronously in the constructor at all.

```
// route config: feature state provided synchronously as part of route providers
export const ORDERS_ROUTES: Routes = [
  {
    path: 'orders',
    providers: [provideState(ordersFeature), provideEffects(OrdersEffects)],
    loadComponent: () => import('./orders.component').then(m => m.OrdersComponent),
  },
];
```

```
// defensive selector composition – never assume another feature's slice exists
export const selectOrdersSummary = createSelector(
  selectOrdersState,
  selectUserPreferencesState, // a different, possibly-not-yet-loaded feature
  (orders, prefs) => ({
    count: orders?.ids.length ?? 0,
    currency: prefs?.currency ?? 'USD',
  })
);
```

```
// component: read via signal, guard rendering, never dereference in the constructor body
export class OrdersComponent {
  private store = inject(Store);
  summary = this.store.selectSignal(selectOrdersSummary); // safe even before full hydration
}
```

I'd also add an integration test that specifically renders the lazy route component in isolation (simulating a hard refresh) with only that feature's providers registered, to catch this class of bug in CI rather than relying on manual "refresh directly to this URL" QA passes.

Interviewer intent: Tests real-world lazy-loading + NgRx feature-registration timing knowledge, a subtle bug class that only surfaces in production-like deep-link scenarios.

Scenario 17: Choosing `createReducer's` `on` immutability correctly for nested arrays

Situation: A reducer case for "reorder columns" in a project-management board does

`state.columns.sort(...)` directly to reorder an array field before returning `{ ...state, columns: state.columns }`. Occasionally the UI doesn't re-render after a reorder, and Redux DevTools shows the action was dispatched but the "diff" pane is empty.

Question: What's wrong, and what's the correct immutable way to reorder an array in a reducer?

Answer: `Array.prototype.sort` mutates the array **in place** and returns the same reference; spreading `{ ...state, columns: state.columns }` afterward still points `columns` at that same (now-reordered) array reference. NgRx/Angular's change-detection and DevTools diffing both rely on reference inequality to detect that something changed — since the array reference never changed, `OnPush` components bound to that slice (and DevTools' diff view, which also compares references for efficiency) see "no change" even though the array's *contents* were mutated. This is a classic immutability violation that produces intermittent-looking bugs because sometimes something else *also* changes in the same dispatch (giving a new top-level state reference) and forces a re-render anyway, masking the root problem until a reorder happens in isolation.

```
// BAD: sort() mutates in place; reference is unchanged
on(BoardActions.reorderColumns, (state, { fromIndex, toIndex }) => {
  const columns = state.columns;
  const [moved] = columns.splice(fromIndex, 1); // mutates!
  columns.splice(toIndex, 0, moved);           // mutates!
  return { ...state, columns };                // same array reference as before
}),
```

```
// GOOD: build a new array immutably
on(BoardActions.reorderColumns, (state, { fromIndex, toIndex }) => {
  const columns = [...state.columns];
  const [moved] = columns.splice(fromIndex, 1); // mutating the COPY is fine
  columns.splice(toIndex, 0, moved);
  return { ...state, columns }; // new array reference
}),
```

I'd also add an ESLint rule (`@ngrx/no-typecast` and general "no mutating array methods in reducers" custom rule, or simply `eslint-plugin-ngrx`'s recommended set) and, more importantly, a unit test asserting `expect(result.columns).not.toBe(initialState.columns)` for every reducer case that touches an array/object field — a cheap, mechanical test that catches this exact class of bug at PR time rather than relying on someone noticing a visual glitch in QA.

Interviewer intent: Tests fundamental understanding of *why* immutability matters mechanically (reference equality driving CD and DevTools), not just "always spread objects" as a memorized rule.

Scenario 18: Effect that needs to read current store state before acting

Situation: A "submit order" effect needs to check the current cart's total against a `maxOrderValue` business rule stored elsewhere in state before calling the API, and dispatch a validation-failure action locally (without an API call) if the rule is violated.

Question: How do you read current store state inside an effect correctly, and what pitfalls should you avoid?

Answer: The correct way is `concatLatestFrom` (from `@ngrx/effects`), which pulls the latest value of a selector *at the moment the triggering action arrives*, without creating a permanent subscription that would leak or cause the effect to re-fire whenever the selected state merely changes (a common mistake using `withLatestFrom(store.select(...))` set up incorrectly, or worse, injecting `Store` and calling `.select().subscribe()` manually inside the effect body).

```
export const submitOrder$ = createEffect(
  (actions$ = inject(Actions), store = inject(Store), api = inject(OrderApi)) =>
    actions$.pipe(
      ofType(OrderActions.submit),
      concatLatestFrom(() => store.select(selectMaxOrderValue)),
      exhaustMap(([{ order }, maxOrderValue]) => {
        if (order.total > maxOrderValue) {
          return of(OrderActions.submitValidationFailed({
            reason: `Order total ${order.total} exceeds limit ${maxOrderValue}`,
          }));
        }
        return api.submit(order).pipe(
          map((confirmed) => OrderActions.submitSuccess({ order: confirmed })),
          catchError((error) => of(OrderActions.submitFailure({ error: error.message })))
        );
      })
    )
);
```

```
);
```

`concatLatestFrom` is preferable to plain `withLatestFrom(store.select(...))` because it lazily subscribes to the selector *only when the source action emits* and unsubscribes immediately after, meaning it evaluates the selector with the state at the correct instant relative to the triggering action, and avoids keeping a long-lived selector subscription alive for the lifetime of the effect that could pick up a "too new" value in edge-case timing scenarios (e.g., if another action races in between and changes `maxOrderValue` before `withLatestFrom`'s already-subscribed stream re-emits). It also composes more readably when combining multiple slices: `concatLatestFrom(() => [store.select(a$), store.select(b$)])`.

I'd flag as a pitfall: never call `store.select(...).pipe(take(1)).toPromise()` or similar ad-hoc subscription patterns inside an effect body outside the RxJS pipeline — it breaks the effect's cancellation semantics (if the outer `actions$` stream tears down, that inner ad-hoc subscription won't be cleaned up by the same mechanism) and is much harder to unit test than a pipeline composed entirely of pipeable operators.

Interviewer intent: Tests knowledge of `concatLatestFrom` specifically and whether the candidate understands why ad-hoc `.subscribe()` calls inside effects are an anti-pattern.

Scenario 19: Deciding between NgRx Store and NgRx Signal Store for a *new* app

Situation: A team starting a greenfield Angular 19 app with heavy use of signals throughout components asks whether to adopt classic `@ngrx/store` (actions/reducers/effects/selectors) or `@ngrx/signals`' Signal Store as the primary state-management approach for the whole app, including complex domains like a multi-step checkout wizard with many interdependent async steps.

Question: What would you recommend, and on what basis — including where you'd still reach for classic Store even in a signals-first app?

Answer: For a signals-first Angular 19+ app, I'd default to `@ngrx/signals`' Signal Store for most feature state — it composes naturally with signal-based components (no `async` pipe indirection), has a smaller API surface (`withState` / `withComputed` / `withMethods` / `rxMethod` covers most needs), and its testing story is simpler (call methods, assert on signals, no action/reducer/effect trio to keep in sync). For most CRUD-shaped and per-feature state — including a checkout wizard's step data — Signal Store with `withEntities` (the signals equivalent of `@ngrx/entity`) is entirely sufficient.

I would still reach for classic `@ngrx/store` (or add it alongside Signal Store) specifically when a domain needs: (a) rigorous auditability via Redux DevTools time-travel across the *whole app* in one unified action log (useful for compliance-heavy domains, e.g., financial approval workflows, where "what sequence of actions led to this state" needs to be inspectable/exportable); (b) genuinely cross-cutting orchestration where many unrelated features must react to the same global event via effects (e.g., "on any 401 response anywhere, log out and clear all feature stores") — classic Store's single global action stream makes this trivial (`actions$.pipe(ofType(anyAction))` observed from one central effects file), whereas coordinating many independent Signal Stores for a truly global concern requires more manual wiring; (c) existing team expertise and a large amount of already-written classic-Store code where a full migration isn't justified (matches Scenario 9's incremental approach).

```
// signal store: default choice for most feature domains, e.g., checkout wizard step state
export const CheckoutStore = signalStore()
```

```

    { providedIn: 'root' },
    withState({ step: 1 as 1 | 2 | 3, shipping: null as ShippingInfo | null, payment: null as
PaymentInfo | null }),
    withComputed(({ step, shipping, payment }) => ({
      canProceed: computed(() => (step() === 1 ? !!shipping() : step() === 2 ? !!payment() :
true)),
    })),
    withMethods((store) => ({
      setShipping: (shipping: ShippingInfo) => patchState(store, { shipping, step: 2 }),
      setPayment: (payment: PaymentInfo) => patchState(store, { payment, step: 3 }),
    }))
  );

// Classic Store: still used for the one truly global, cross-cutting concern
export const globalAuthEffects = createEffect(
  (actions$ = inject(Actions), router = inject(Router)) =>
    actions$.pipe(
      ofType(HttpActions.unauthorized), // dispatched by an HTTP interceptor for ANY 401,
      any feature
      tap(() => router.navigate(['/login']))
    ),
  { functional: true, dispatch: false }
);

```

My honest recommendation for this specific greenfield app: Signal Store for feature/domain state (including checkout), classic Store kept minimal and reserved for genuinely global cross-cutting action-driven concerns (auth, global error handling, analytics-on-every-action) — not because the two can't both do everything, but because using each where it's most natural keeps both simpler than forcing one paradigm to cover everything.

Interviewer intent: Tests whether the candidate has an opinionated, experience-backed answer about the two paradigms' actual tradeoffs rather than treating Signal Store as a strict drop-in replacement for every use of classic Store.

Scenario 20: `withEntities` in Signal Store vs. classic `@ngrx/entity`

Situation: Following the Scenario 19 decision, the team migrates the API-keys admin table (Scenario 11) to Signal Store and wants the same CRUD ergonomics `createEntityAdapter` provided, but expressed in signals idiom.

Question: Rewrite the API-keys feature using `@ngrx/signals'` `withEntities`, and compare its ergonomics to the classic entity adapter.

Answer: `withEntities` (from `@ngrx/signals/entities`) provides the signals-world equivalent of `createEntityAdapter` — normalized `entityMap/ids` state plus updater functions (`addEntity`, `updateEntity`, `removeEntity`, `setAllEntities`, `updateEntities` for bulk) and matching entity selectors as computed signals, following the same normalized-storage philosophy as Scenario 4 and Scenario 11.

```

interface ApiKey { id: string; name: string; revoked: boolean; createdAt: string; }

```



```

export const ApiKeysStore = signalStore(
  { providedIn: 'root' },
  withEntities<ApiKey>(),
  withState({ selectedIds: [] as string[], loading: false }),
  withComputed(({ entities, ids, selectedIds }) => ({
    allKeys: computed(() => ids().map(id => entities()[id])),
    selectedKeys: computed(() => selectedIds().map(id => entities()[id]).filter(Boolean)),
  })),
  withMethods((store, api = inject(ApiKeysApi)) => ({
    load: rxMethod<void>(
      pipe(
        tap(() => patchState(store, { loading: true })),
        switchMap(() =>
          api.list().pipe(
            tapResponse({
              next: (keys) => patchState(store, setAllEntities(keys), { loading: false }),
              error: () => patchState(store, { loading: false }),
            })
          )
        )
      ),
    toggleSelected(id: string) {
      patchState(store, (state) => ({
        selectedIds: state.selectedIds.includes(id)
          ? state.selectedIds.filter(x => x !== id)
          : [...state.selectedIds, id],
      }));
    },
    bulkRevoke: rxMethod<string[]>(
      pipe(
        exhaustMap((ids) =>
          api.revokeMany(ids).pipe(
            tapResponse({
              next: () => patchState(
                store,
                updateEntities({ ids, changes: { revoked: true } }),
                { selectedIds: [] }
              ),
              error: () => {},
            })
          )
        )
      ),
    ),
  })),
);

```

Ergonomic comparison: classic `createEntityAdapter` requires wiring adapter functions into a hand-written reducer's `on(...)` cases and separately calling `.getSelectors()` to expose read access — two distinct mechanical steps, in two different NgRx sub-APIs (`@ngrx/store` + `@ngrx/entity`). `withEntities` collapses this: the updater functions (`setAllEntities`, `updateEntities`, etc.) are passed straight to `patchState` inside

`withMethods`, and reads are already-reactive computed signals (`entities`, `ids`) with no separate selector-registration step. The tradeoff is ecosystem maturity — classic `@ngrx/entity` has years of production hardening and broad community Stack Overflow coverage for edge cases (e.g., custom `sortComparer` performance at scale), while `@ngrx/signals`' entity API, though stable, is newer and the team should budget a bit more time for hitting undocumented edge cases first-hand. For a genuinely new feature in a signals-first app, I'd still choose `withEntities` — the ergonomic and mental-model consistency with the rest of the signal-based codebase outweighs the maturity gap for typical CRUD needs.

Interviewer intent: Tests whether the candidate has hands-on familiarity with the newest `@ngrx/signals/entities` API, not just the classic `@ngrx/entity` adapter, and can articulate real (not hand-wavy) tradeoffs between them.

Quick Revision Cheat Sheet

- **Don't default to NgRx.** Reach for it when state is genuinely shared across unrelated parts of the tree, needs complex async orchestration, or requires audit/time-travel — not for a single self-contained widget (a signal-based service is often enough).
- **Memoization is about input-selector reference equality, not "selectors are magically cached."** Keep input selectors as plain slice reads; put all computation in the outer projector. Inline `.filter()` / `.map()` in an input selector breaks memoization.
- **Effects loop when input and output action types overlap.** Always separate command actions (`update`) from result actions (`updateSuccess` / `updateFailure`) via `createActionGroup`.
- **Normalize relational state** (flat entity tables keyed by ID, references by ID only) instead of nesting/denormalizing — avoids fan-out updates and keeps each entity's mutations isolated and testable.
- **Optimistic updates must carry their own rollback data** in the failure action (not look it up from "current" state later), and pick the right flattening operator: `concatMap` to preserve order, `switchMap` to cancel stale requests, `exhaustMap` to ignore duplicate triggers.
- **Test reducers as pure functions** (no TestBed), **test selectors via** `.projector` for correctness and via input-reference stability for memoization, and **reserve marble testing for effects where timing itself is the assertion** (debounce, race) — simpler `provideMockActions` + plain observables suffice otherwise.
- `@ngrx/signals` **Signal Store coexists with classic** `@ngrx/store` — migrate feature-by-feature; keep classic Store for genuinely global, cross-cutting, action-log-driven concerns (auth, global error handling) if you keep it at all.
- **Redux DevTools time-travel turns intermittent bugs into static, steppable artifacts** — diff state before/after each action to localize a bug to reducer vs. selector vs. component.
- `@ngrx/entity`'s `createEntityAdapter` (**classic**) and `withEntities` (**Signal Store**) both give O(1) lookups and safe bulk CRUD helpers — always prefer them over manual array `findIndex` / splice logic for CRUD-heavy state.
- **Never mutate arrays/objects in place in a reducer** (`sort`, `push`, `splice` on the original reference) — reference equality is what drives change detection and DevTools diffing; test with `expect(result.field).not.toBe(previous.field)`.
- **Derived state belongs in selectors, not in stored state fields** — if a field can be recomputed from the

rest of state, storing it separately guarantees eventual drift bugs.

- **Use `concatLatestFrom` to read current store state inside an effect**, never ad-hoc `.subscribe()` / `.toPromise()` calls, to preserve correct timing and cancellation semantics.
- `createActionGroup` and `createFeature` are the current idiomatic way to define actions and reducer+selectors respectively — know them, don't only know the older verbose `createAction/createFeatureSelector` boilerplate style.
- **Parameterized selector factories must be instantiated once and reused**, not called fresh inline in a template or computed — otherwise each call creates a new memoization cache and defeats the purpose entirely.

Created By - Durgesh Singh